

Spanner: Google's Globally Distributed Database

JAMES C. CORBETT, JEFFREY DEAN, MICHAEL EPSTEIN, ANDREW FIKES, CHRISTOPHER FROST, J. J. FURMAN, SANJAY GHEMAWAT, ANDREY GUBAREV, CHRISTOPHER HEISER, PETER HOCHSCHILD, WILSON HSIEH, SEBASTIAN KANTHAK, EUGENE KOGAN, HONGYI LI, ALEXANDER LLOYD, SERGEY MELNIK, DAVID MWAURA, DAVID NAGLE, SEAN QUINLAN, RAJESH RAO, LINDSAY ROLIG, YASUSHI SAITO, MICHAL SZYMANIAK, CHRISTOPHER TAYLOR, RUTH WANG, and DALE WOODFORD, Google, Inc.

Spanner is Google's scalable, multiversion, globally distributed, and synchronously replicated database. It is the first system to distribute data at global scale and support externally-consistent distributed transactions. This article describes how Spanner is structured, its feature set, the rationale underlying various design decisions, and a novel time API that exposes clock uncertainty. This API and its implementation are critical to supporting external consistency and a variety of powerful features: nonblocking reads in the past, lock-free snapshot transactions, and atomic schema changes, across all of Spanner.

Categories and Subject Descriptors: H.2.4 [Database Management]: Systems—Concurrency, distributed databases, transaction processing

General Terms: Design, Algorithms, Performance

Additional Key Words and Phrases: Distributed databases, concurrency control, replication, transactions, time management

ACM Reference Format:

Corbett, J. C., Dean, J., Epstein, M., Fikes, A., Frost, C., Furman, J. J., Ghemawat, S., Gubarev, A., Heiser, C., Hochschild, P., Hsieh, W., Kanthak, S., Kogan, E., Li, H., Lloyd, A., Melnik, S., Mwaura, D., Nagle, D., Quinlan, S., Rao, R., Rolig, L., Saito, Y., Szymaniak, M., Taylor, C., Wang, R., and Woodford, D. 2013. Spanner: Google's globally distributed database. *ACM Trans. Comput. Syst.* 31, 3, Article 8 (August 2013), 22 pages.

DOI: <http://dx.doi.org/10.1145/2491245>

1. INTRODUCTION

Spanner is a scalable, globally distributed database designed, built, and deployed at Google. At the highest level of abstraction, it is a database that shards data across many sets of Paxos [Lamport 1998] state machines in datacenters spread all over the world. Replication is used for global availability and geographic locality; clients automatically failover between replicas. Spanner automatically reshards data across machines as the amount of data or the number of servers changes, and it automatically migrates data across machines (even across datacenters) to balance load and in

This article is essentially the same (with minor reorganizations, corrections, and additions) as the paper of the same title that appeared in the Proceedings of OSDI 2012.

Authors' address: J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild, W. Hsieh (corresponding author), S. Kanthak, E. Kogan, H. Li, A. Lloyd, S. Melnik, D. Mwaura, D. Nagle, S. Quinlan, R. Rao, L. Rolig, Y. Saito, M. Szymaniak, C. Taylor, R. Wang, and D. Woodford, Google, Inc. 1600 Amphitheatre Parkway, Mountain View, CA 94043; email: wilsonh@google.com.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses contact the Owner/Author.

2013 Copyright is held by the author/owner(s).

0734-2071/2013/08-ART8

DOI: <http://dx.doi.org/10.1145/2491245>

response to failures. Spanner is designed to scale up to millions of machines across hundreds of datacenters and trillions of database rows.

Applications can use Spanner for high availability, even in the face of wide-area natural disasters, by replicating their data within or even across continents. Our initial customer was F1 [Shute et al. 2012], a rewrite of Google's advertising backend. F1 uses five replicas spread across the United States. Most other applications will probably replicate their data across 3 to 5 datacenters in one geographic region, but with relatively independent failure modes. That is, most applications will choose lower latency over higher availability, as long as they can survive 1 or 2 datacenter failures.

Spanner's main focus is managing cross-datacenter replicated data, but we have also spent a great deal of time in designing and implementing important database features on top of our distributed-systems infrastructure. Even though many projects happily use Bigtable [Chang et al. 2008], we have also consistently received complaints from users that Bigtable can be difficult to use for some kinds of applications: those that have complex, evolving schemas, or those that want strong consistency in the presence of wide-area replication. (Similar claims have been made by other authors [Stonebraker 2010b].) Many applications at Google have chosen to use Megastore [Baker et al. 2011] because of its semirelational data model and support for synchronous replication, despite its relatively poor write throughput. As a consequence, Spanner has evolved from a Bigtable-like versioned key-value store into a temporal multiversion database. Data is stored in schematized semirelational tables; data is versioned, and each version is automatically timestamped with its commit time; old versions of data are subject to configurable garbage-collection policies; and applications can read data at old timestamps. Spanner supports general-purpose transactions, and provides an SQL-based query language.

As a globally distributed database, Spanner provides several interesting features. First, the replication configurations for data can be dynamically controlled at a fine grain by applications. Applications can specify constraints to control which datacenters contain which data, how far data is from its users (to control read latency), how far replicas are from each other (to control write latency), and how many replicas are maintained (to control durability, availability, and read performance). Data can also be dynamically and transparently moved between datacenters by the system to balance resource usage across datacenters. Second, Spanner has two features that are difficult to implement in a distributed database: it provides externally consistent [Gifford 1982] reads and writes, and globally consistent reads across the database at a timestamp. These features enable Spanner to support consistent backups, consistent MapReduce executions [Dean and Ghemawat 2010], and atomic schema updates, all at global scale, and even in the presence of ongoing transactions.

These features are enabled by the fact that Spanner assigns globally meaningful commit timestamps to transactions, even though transactions may be distributed. The timestamps reflect serialization order. In addition, the serialization order satisfies external consistency (or equivalently, linearizability [Herlihy and Wing 1990]): if a transaction T_1 commits before another transaction T_2 starts, then T_1 's commit timestamp is smaller than T_2 's. Spanner is the first system to provide such guarantees at global scale.

The key enabler of these properties is a new TrueTime API and its implementation. The API directly exposes clock uncertainty, and the guarantees on Spanner's timestamps depend on the bounds that the implementation provides. If the uncertainty is large, Spanner slows down to wait out that uncertainty. Google's cluster-management software provides an implementation of the TrueTime API. This implementation keeps uncertainty small (generally less than 10ms) by using multiple modern clock references (GPS and atomic clocks). Conservatively reporting

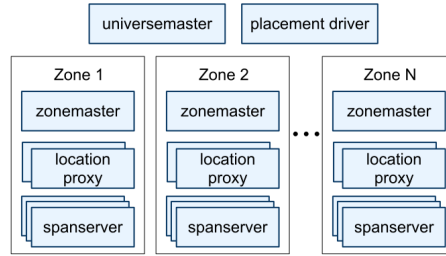


Fig. 1. Spanner server organization.

uncertainty is necessary for correctness; keeping the bound on uncertainty small is necessary for performance.

Section 2 describes the structure of Spanner's implementation, its feature set, and the engineering decisions that went into their design. Section 3 describes our new TrueTime API and sketches its implementation. Section 4 describes how Spanner uses TrueTime to implement externally-consistent distributed transactions, lock-free snapshot transactions, and atomic schema updates. Section 5 provides some benchmarks on Spanner's performance and TrueTime behavior, and discusses the experiences of F1. Sections 6, 7, and 8 describe related and future work, and summarize our conclusions.

2. IMPLEMENTATION

This section describes the structure of and rationale underlying Spanner's implementation. It then describes the *directory* abstraction, which is used to manage replication and locality, and is the unit of data movement. Finally, it describes our data model, why Spanner looks like a relational database instead of a key-value store, and how applications can control data locality.

A Spanner deployment is called a *universe*. Given that Spanner manages data globally, there will be only a handful of running universes. We currently run a test/playground universe, a development/production universe, and a production-only universe.

Spanner is organized as a set of *zones*, where each zone is the rough analog of a deployment of Bigtable servers [Chang et al. 2008]. Zones are the unit of administrative deployment. The set of zones is also the set of locations across which data can be replicated. Zones can be added to or removed from a running system as new datacenters are brought into service and old ones are turned off, respectively. Zones are also the unit of physical isolation: there may be one or more zones in a datacenter, for example, if different applications' data must be partitioned across different sets of servers in the same datacenter.

Figure 1 illustrates the servers in a Spanner universe. A zone has one *zonemaster* and between one hundred and several thousand *spanservers*. The former assigns data to spanservers; the latter serve data to clients. The per-zone *location proxies* are used by clients to locate the spanservers assigned to serve their data. The *universe master* and the *placement driver* are currently singletons. The universe master is primarily a console that displays status information about all the zones for interactive debugging. The placement driver handles automated movement of data across zones on the timescale of minutes. The placement driver periodically communicates with the spanservers to find data that needs to be moved, either to meet updated replication constraints or to balance load. For space reasons, we will only describe the spanserver in any detail.

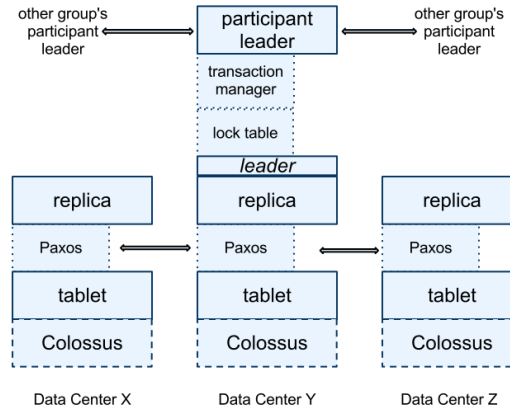


Fig. 2. Spanserver software stack.

2.1. Spanserver Software Stack

This section focuses on the spanserver implementation to illustrate how replication and distributed transactions have been layered onto our Bigtable-based implementation. The software stack is shown in Figure 2. At the bottom, each spanserver is responsible for between 100 and 1000 instances of a data structure called a *tablet*. A tablet is similar to Bigtable’s tablet abstraction, in that it implements a bag of the following mappings.

(key:string, timestamp:int64) → string

Unlike Bigtable, Spanner assigns timestamps to data, which is an important way in which Spanner is more like a multiversion database than a key-value store. A tablet’s state is stored in a set of B-tree-like files and a write-ahead log, all on a distributed file system called Colossus (the successor to the Google File System [Ghemawat et al. 2003]).

To support replication, each spanserver implements a single Paxos state machine on top of each tablet. (An early Spanner incarnation supported multiple Paxos state machines per tablet, which allowed for more flexible replication configurations. The complexity of that design led us to abandon it.) Each state machine stores its meta-data and log in its corresponding tablet. Our Paxos implementation supports long-lived leaders with time-based leader leases, whose length defaults to 10 seconds. The current Spanner implementation logs every Paxos write twice: once in the tablet’s log, and once in the Paxos log. This choice was made out of expediency, and we are likely to remedy this eventually. Our implementation of Paxos is pipelined, so as to improve Spanner’s throughput in the presence of WAN latencies. By “pipelined,” we mean Lamport’s “multi-decree parliament” [Lamport 1998], which both amortizes the cost of electing a leader across multiple decrees and allows for concurrent voting on different decrees. It is important to note that although decrees may be approved out of order, the decrees are applied in order (a fact on which we will depend in Section 4).

The Paxos state machines are used to implement a consistently replicated bag of mappings. The key-value mapping state of each replica is stored in its corresponding tablet. Writes must initiate the Paxos protocol at the leader; reads access state directly from the underlying tablet at any replica that is sufficiently up-to-date. The set of replicas is collectively a *Paxos group*.

At every replica that is a leader, a spanserver implements a *lock table* to implement concurrency control. The lock table contains the state for two-phase locking: it maps

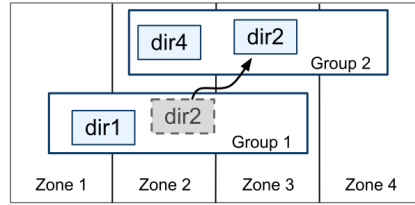


Fig. 3. Directories are the unit of data movement between Paxos groups.

ranges of keys to lock states. (Note that having a long-lived Paxos leader is critical to efficiently managing the lock table.) In both Bigtable and Spanner, we designed for long-lived transactions (for example, for report generation, which might take on the order of minutes), which perform poorly under optimistic concurrency control in the presence of conflicts. Operations that require synchronization, such as transactional reads, acquire locks in the lock table; other operations bypass the lock table. The state of the lock table is mostly volatile (i.e., not replicated via Paxos): we explain the details further in Section 4.2.1.

At every replica that is a leader, each spanserver also implements a *transaction manager* to support distributed transactions. The transaction manager is used to implement a *participant leader*; the other replicas in the group will be referred to as *participant slaves*. If a transaction involves only one Paxos group (as is the case for most transactions), it can bypass the transaction manager, since the lock table and Paxos together provide transactionality. If a transaction involves more than one Paxos group, those groups' leaders coordinate to perform two-phase commit. One of the participant groups is chosen as the coordinator: the participant leader of that group will be referred to as the *coordinator leader*, and the slaves of that group as *coordinator slaves*. The state of each transaction manager is stored in the underlying Paxos group (and therefore is replicated).

2.2. Directories and Placement

On top of the bag of key-value mappings, the Spanner implementation supports a bucketing abstraction called a *directory*, which is a set of contiguous keys that share a common prefix. (The choice of the term *directory* is a historical accident; a better term might be *bucket*.) We will explain the source of that prefix in Section 2.3. Supporting directories allows applications to control the locality of their data by choosing keys carefully.

A directory is the unit of data placement. All data in a directory has the same replication configuration. When data is moved between Paxos groups, it is moved directory by directory, as shown in Figure 3. Spanner might move a directory to shed load from a Paxos group; to put directories that are frequently accessed together into the same group; or to move a directory into a group that is closer to its accessors. Directories can be moved while client operations are ongoing. One would expect that a 50MB directory could be moved in a few seconds.

The fact that a Paxos group may contain multiple directories implies that a Spanner tablet is different from a Bigtable tablet: the former is not necessarily a single lexicographically contiguous partition of the row space. Instead, a Spanner tablet is a container that may encapsulate multiple partitions of the row space. We made this decision so that it would be possible to colocate multiple directories that are frequently accessed together.

Movedir is the background task used to move directories between Paxos groups [Douceur and Howell 2006]. Movedir is also used to add to or remove replicas

from Paxos groups [Lorch et al. 2006] by moving all of a group's data to a new group with the desired configuration, because Spanner does not yet support in-Paxos configuration changes. Movedir is not implemented as a single transaction, so as to avoid blocking ongoing reads and writes on a bulky data move. Instead, movedir registers the fact that it is starting to move data and moves the data in the background. When it has moved all but a nominal amount of the data, it uses a transaction to atomically move that nominal amount and update the metadata for the two Paxos groups.

A directory is also the smallest unit whose geographic-replication properties (or *placement*, for short) can be specified by an application. The design of our placement-specification language separates responsibilities for managing replication configurations. Administrators control two dimensions: the number and types of replicas, and the geographic placement of those replicas. They create a menu of named options in these two dimensions (e.g., *North America, replicated 5 ways with 1 witness*). An application controls how data is replicated, by tagging each database and/or individual directories with a combination of those options. For example, an application might store each end-user's data in its own directory, which would enable user *A*'s data to have three replicas in Europe, and user *B*'s data to have five replicas in North America.

For expository clarity we have oversimplified. In fact, Spanner will shard a directory into multiple *fragments* if it grows too large. Fragments may be served from different Paxos groups (and therefore different servers). Movedir actually moves fragments, and not whole directories, between groups.

2.3. Data Model

Spanner exposes the following set of data features to applications: a data model based on schematized semirelational tables, a query language, and general-purpose transactions. The move towards supporting these features was driven by many factors. The need to support schematized semirelational tables and synchronous replication is supported by the popularity of Megastore [Baker et al. 2011]. At least 300 applications within Google use Megastore (despite its relatively low performance) because its data model is simpler to manage than Bigtable's, and because of its support for synchronous replication across datacenters. (Bigtable only supports eventually-consistent replication across datacenters.) Examples of well-known Google applications that use Megastore are Gmail, Picasa, Calendar, Android Market, and AppEngine. The need to support an SQL-like query language in Spanner was also clear, given the popularity of Dremel [Melnik et al. 2010] as an interactive data-analysis tool. Finally, the lack of cross-row transactions in Bigtable led to frequent complaints; Percolator [Peng and Dabek 2010] was in part built to address this failing. Some authors have claimed that general two-phase commit is too expensive to support, because of the performance or availability problems that it brings [Chang et al. 2008; Cooper et al. 2008; Helland 2007]. We believe it is better to have application programmers deal with performance problems due to overuse of transactions as bottlenecks arise, rather than always coding around the lack of transactions. Running two-phase commit over Paxos mitigates the availability problems.

The application data model is layered on top of the directory-bucketed key-value mappings supported by the implementation. An application creates one or more databases in a universe. Each database can contain an unlimited number of schematized tables. Tables look like relational-database tables, with rows, columns, and versioned values. We will not go into detail about the query language for Spanner. It looks like SQL with some extensions to support protocol-buffer-valued [Google 2008] fields.

Spanner's data model is not purely relational, in that rows must have names. More precisely, every table is required to have an ordered set of one or more primary-key columns. This requirement is where Spanner still looks like a key-value store: the

```
CREATE TABLE Users {
  uid INT64 NOT NULL, email STRING
} PRIMARY KEY (uid), DIRECTORY;

CREATE TABLE Albums {
  uid INT64 NOT NULL, aid INT64 NOT NULL,
  name STRING
} PRIMARY KEY (uid, aid),
  INTERLEAVE IN PARENT Users ON DELETE CASCADE;
```

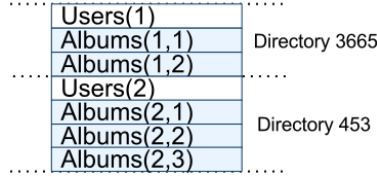


Fig. 4. Example Spanner schema for photo metadata, and the interleaving implied by `INTERLEAVE IN`.

Table I. TrueTime API. The Argument t is of Type $TTstamp$

Method	Returns
$TT.now()$	$TTinterval$: $[earliest, latest]$
$TT.after(t)$	true if t has definitely passed
$TT.before(t)$	true if t has definitely not arrived

primary keys form the name for a row, and each table defines a mapping from the primary-key columns to the non-primary-key columns. A row has existence only if some value (even if it is `NULL`) is defined for the row's keys. Imposing this structure is useful because it lets applications control data locality through their choices of keys.

Figure 4 contains an example Spanner schema for storing photo metadata on a per-user, per-album basis. The schema language is similar to Megastore's, with the additional requirement that every Spanner database must be partitioned by clients into one or more hierarchies of tables. Client applications declare the hierarchies in database schemas via the `INTERLEAVE IN` declarations. The table at the top of a hierarchy is a *directory table*. Each row in a directory table with key K , together with all of the rows in descendant tables that start with K in lexicographic order, forms a directory. `ON DELETE CASCADE` says that deleting a row in the directory table deletes any associated child rows. The figure also illustrates the interleaved layout for the example database: for example, `Albums(2,1)` represents the row from the `Albums` table for `user_id 2`, `album_id 1`. This interleaving of tables to form directories is significant because it allows clients to describe the locality relationships that exist between multiple tables, which is necessary for good performance in a sharded, distributed database. Without it, Spanner would not know the most important locality relationships.

3. TRUETIME

This section describes the TrueTime API and sketches its implementation. We leave most of the details for another article: our goal is to demonstrate the power of having such an API. Table I lists the methods of the API. TrueTime explicitly represents time as a $TTinterval$, which is an interval with bounded time uncertainty (unlike standard time interfaces that give clients no notion of uncertainty). The endpoints of a $TTinterval$ are of type $TTstamp$. The $TT.now()$ method returns a $TTinterval$ that is guaranteed to contain the absolute time during which $TT.now()$ was invoked. The time epoch is analogous to UNIX time with leap-second smearing. Define the instantaneous

error bound as ϵ , which is half of the interval's width, and the average error bound as $\bar{\epsilon}$. The *TT.after()* and *TT.before()* methods are convenience wrappers around *TT.now()*.

Denote the absolute time of an event e by the function $t_{abs}(e)$. In more formal terms, TrueTime guarantees that for an invocation $tt = TT.now()$, $tt.earliest \leq t_{abs}(e_{now}) \leq tt.latest$, where e_{now} is the invocation event.

The underlying time references used by TrueTime are GPS and atomic clocks. TrueTime uses two forms of time reference because they have different failure modes. GPS reference-source vulnerabilities include antenna and receiver failures, local radio interference, correlated failures (e.g., design faults such as incorrect leap-second handling and spoofing), and GPS system outages. Atomic clocks can fail in ways uncorrelated to GPS and each other, and over long periods of time can drift significantly due to frequency error.

TrueTime is implemented by a set of *time master* machines per datacenter and a *timeslave daemon* per machine. The majority of masters have GPS receivers with dedicated antennas; these masters are separated physically to reduce the effects of antenna failures, radio interference, and spoofing. The remaining masters (which we refer to as *Armageddon masters*) are equipped with atomic clocks. An atomic clock is not that expensive: the cost of an Armageddon master is of the same order as that of a GPS master. All masters' time references are regularly compared against each other. Each master also cross-checks the rate at which its reference advances time against its own local clock, and evicts itself if there is substantial divergence. Between synchronizations, Armageddon masters advertise a slowly increasing time uncertainty that is derived from conservatively applied worst-case clock drift. GPS masters advertise uncertainty that is typically close to zero.

Every daemon polls a variety of masters [Mills 1981] to reduce vulnerability to errors from any one master. Some are GPS masters chosen from nearby datacenters; the rest are GPS masters from farther datacenters, as well as some Armageddon masters. Daemons apply a variant of Marzullo's algorithm [Marzullo and Owicki 1983] to detect and reject liars, and synchronize the local machine clocks to the non-liars. To protect against broken local clocks, machines that exhibit frequency excursions larger than the worst-case bound derived from component specifications and operating environment are evicted. Correctness depends on ensuring that the worst-case bound is enforced.

Between synchronizations, a daemon advertises a slowly increasing time uncertainty. ϵ is derived from conservatively applied worst-case local clock drift. ϵ also depends on time-master uncertainty and communication delay to the time masters. In our production environment, ϵ is typically a sawtooth function of time, varying from about 1 to 7 ms over each poll interval. $\bar{\epsilon}$ is therefore 4 ms most of the time. The daemon's poll interval is currently 30 seconds, and the current applied drift rate is set at 200 microseconds/second, which together account for the sawtooth bounds from 0 to 6 ms. The remaining 1 ms comes from the communication delay to the time masters. Excursions from this sawtooth are possible in the presence of failures. For example, occasional time-master unavailability can cause datacenter-wide increases in ϵ . Similarly, overloaded machines and network links can result in occasional localized ϵ spikes. Correctness is not affected by ϵ variance because Spanner can wait out the uncertainty, but performance can degrade if ϵ increases too much.

4. CONCURRENCY CONTROL

This section describes how TrueTime is used to guarantee the correctness properties around concurrency control, and how those properties are used to implement features such as externally consistent transactions, lock-free snapshot transactions, and nonblocking reads in the past. These features enable, for example, the guarantee that

Table II. Types of Reads and Writes in Spanner, and How They Compare

Operation	Timestamp Discussion	Concurrency Control	Replica Required
Read-Write Transaction	§ 4.1.2	pessimistic	leader
Snapshot Transaction	§ 4.1.4	lock-free	leader for timestamp; any for read, subject to § 4.1.3
Snapshot Read, client-chosen timestamp	—	lock-free	any, subject to § 4.1.3
Snapshot Read, client-chosen bound	§ 4.1.3	lock-free	any, subject to § 4.1.3

a whole-database audit read at a timestamp t will see exactly the effects of every transaction that has committed as of t .

Going forward, it will be important to distinguish writes as seen by Paxos (which we will refer to as *Paxos writes* unless the context is clear) from Spanner client writes. For example, two-phase commit generates a Paxos write for the prepare phase that has no corresponding Spanner client write.

4.1. Timestamp Management

Table II lists the types of operations that Spanner supports. The Spanner implementation supports *read-write transactions*, *snapshot transactions* (predeclared snapshot-isolation transactions), and *snapshot reads*. Standalone writes are implemented as read-write transactions; non-snapshot standalone reads are implemented as snapshot transactions. Both are internally retried (clients need not write their own retry loops).

A snapshot transaction is a kind of transaction that has the performance benefits of snapshot isolation [Berenson et al. 1995]. A snapshot transaction must be predeclared as not having any writes; it is not simply a read-write transaction without any writes. Reads in a snapshot transaction execute at a system-chosen timestamp without locking, so that incoming writes are not blocked. The execution of the reads in a snapshot transaction can proceed on any replica that is sufficiently up-to-date (Section 4.1.3).

A snapshot read is a read in the past that executes without locking. A client can either specify a timestamp for a snapshot read, or provide an upper bound on the desired timestamp's staleness and let Spanner choose a timestamp. In either case, the execution of a snapshot read proceeds at any replica that is sufficiently up-to-date.

For both snapshot transactions and snapshot reads, commit is inevitable once a timestamp has been chosen, unless the data at that timestamp has been garbage-collected. As a result, clients can avoid buffering results inside a retry loop. When a server fails, clients can internally continue the query on a different server by repeating the timestamp and the current read position.

4.1.1. Paxos Leader Leases. Spanner's Paxos implementation uses timed leases to make leadership long-lived (10 seconds by default). A potential leader sends requests for timed *lease votes*; upon receiving a quorum of lease votes the leader knows it has a lease. A replica extends its lease vote implicitly on a successful write, and the leader requests lease-vote extensions if they are near expiration. Define a leader's *lease interval* to start when it discovers it has a quorum of lease votes, and to end when it no longer has a quorum of lease votes (because some have expired). Spanner depends on the following disjointness invariant: for each Paxos group, each Paxos leader's lease interval is disjoint from every other leader's. Section 4.2.5 describes how this invariant is enforced.

The Spanner implementation permits a Paxos leader to abdicate by releasing its slaves from their lease votes. To preserve the disjointness invariant, Spanner

constrains when abdication is permissible. Define s_{max} to be the maximum timestamp used by a leader. Subsequent sections will describe when s_{max} is advanced. Before abdicating, a leader must wait until $TT.after(s_{max})$ is true.

4.1.2. Assigning Timestamps to RW Transactions. Transactional reads and writes use strict two-phase locking. As a result, they can be assigned timestamps at any time when all locks have been acquired, but before any locks have been released. For a given transaction, Spanner assigns it the timestamp that Paxos assigns to the Paxos write that represents the transaction commit.

Spanner depends on the following monotonicity invariant: within each Paxos group, Spanner assigns timestamps to Paxos writes in monotonically increasing order, even across leaders. A single leader replica can trivially assign timestamps in monotonically increasing order. This invariant is enforced across leaders by making use of the disjointness invariant: a leader must only assign timestamps within the interval of its leader lease. Note that whenever a timestamp s is assigned, s_{max} is advanced to s to preserve disjointness.

Spanner also enforces the following external-consistency invariant: if the start of a transaction T_2 occurs after the commit of a transaction T_1 , then the commit timestamp of T_2 must be greater than the commit timestamp of T_1 . Define the start and commit events for a transaction T_i by e_i^{start} and e_i^{commit} ; and the commit timestamp of a transaction T_i by s_i . The invariant becomes $t_{abs}(e_1^{commit}) < t_{abs}(e_2^{start}) \Rightarrow s_1 < s_2$. The protocol for executing transactions and assigning timestamps obeys two rules, which together guarantee this invariant, as shown in the following. Define the arrival event of the commit request at the coordinator leader for a write T_i to be e_i^{server} .

Start. The coordinator leader for a write T_i assigns a commit timestamp s_i no less than the value of $TT.now().latest$, computed after e_i^{server} . Note that the participant leaders do not matter here; Section 4.2.1 describes how they are involved in the implementation of the next rule.

Commit Wait. The coordinator leader ensures that clients cannot see any data committed by T_i until $TT.after(s_i)$ is true. Commit wait ensures that s_i is less than the absolute commit time of T_i , or $s_i < t_{abs}(e_i^{commit})$. The implementation of commit wait is described in Section 4.2.1. Proof:

$$\begin{array}{ll}
 s_1 < t_{abs}(e_1^{commit}) & \text{(commit wait)} \\
 t_{abs}(e_1^{commit}) < t_{abs}(e_2^{start}) & \text{(assumption)} \\
 t_{abs}(e_2^{start}) \leq t_{abs}(e_2^{server}) & \text{(causality)} \\
 t_{abs}(e_2^{server}) \leq s_2 & \text{(start)} \\
 s_1 < s_2 & \text{(transitivity)}
 \end{array}$$

4.1.3. Serving Reads at a Timestamp. The monotonicity invariant described in Section 4.1.2 allows Spanner to correctly determine whether a replica's state is sufficiently up-to-date to satisfy a read. Every replica tracks a value called *safe time*, t_{safe} , which is the maximum timestamp at which a replica is up-to-date. A replica can satisfy a read at a timestamp t if $t \leq t_{safe}$.

Define $t_{safe} = \min(t_{safe}^{Paxos}, t_{safe}^{TM})$, where each Paxos state machine has a safe time t_{safe}^{Paxos} and each transaction manager has a safe time t_{safe}^{TM} . t_{safe}^{Paxos} is simpler: it is the timestamp of the highest-applied Paxos write. Because timestamps increase monotonically and

writes are applied in order, writes will no longer occur at or below t_{safe}^{Paxos} with respect to Paxos.

t_{safe}^{TM} is ∞ at a replica if there are zero prepared (but not committed) transactions—that is, transactions in between the two phases of two-phase commit. (For a participant slave, t_{safe}^{TM} actually refers to the replica's leader's transaction manager, whose state the slave can infer through metadata passed on Paxos writes.) If there are any such transactions, then the state affected by those transactions is indeterminate: a participant replica does not yet know whether such transactions will commit. As we discuss in Section 4.2.1, the commit protocol ensures that every participant knows a lower bound on a prepared transaction's timestamp. Every participant leader (for a group g) for a transaction T_i assigns a prepare timestamp $s_{i,g}^{prepare}$ to its prepare record. The coordinator leader ensures that the transaction's commit timestamp $s_i \geq s_{i,g}^{prepare}$ over all participant groups g . Therefore, for every replica in a group g , over all transactions T_i prepared at g , $t_{safe}^{TM} = \min_i(s_{i,g}^{prepare}) - 1$ over all transactions prepared at g .

4.1.4. Assigning Timestamps to RO Transactions. A snapshot transaction executes in two phases: assign a timestamp s_{read} [Chan and Gray 1985], and then execute the transaction's reads as snapshot reads at s_{read} . The snapshot reads can execute at any replicas that are sufficiently up-to-date.

The simple assignment of $s_{read} = TT.now().latest$, at any time after a transaction starts, preserves external consistency by an argument analogous to that presented for writes in Section 4.1.2. However, such a timestamp may require the execution of the data reads at s_{read} to block if t_{safe} has not advanced sufficiently. (In addition, note that choosing a value of s_{read} may also advance s_{max} to preserve disjointness.) To reduce the chances of blocking, Spanner should assign the oldest timestamp that preserves external consistency. Section 4.2.2 explains how such a timestamp can be chosen.

4.2. Details

This section explains some of the practical details of read-write transactions and snapshot transactions elided earlier, as well as the implementation of a special transaction type used to implement atomic schema changes. It then describes some refinements of the basic schemes as described.

4.2.1. Read-Write Transactions. Like Bigtable, writes that occur in a transaction are buffered at the client until commit. As a result, reads in a transaction do not see the effects of the transaction's writes. This design works well in Spanner because a read returns the timestamps of any data read, and uncommitted writes have not yet been assigned timestamps.

Reads within read-write transactions use wound-wait [Rosenkrantz et al. 1978] to avoid deadlocks. The client issues reads to the leader replica of the appropriate group, which acquires read locks and then reads the most recent data. While a client transaction remains open, it sends keepalive messages to prevent participant leaders from timing out its transaction. When a client has completed all reads and buffered all writes, it begins two-phase commit. The client chooses a coordinator group and sends a commit message to each participant's leader with the identity of the coordinator and any buffered writes. Having the client drive the two-phase commit protocol avoids sending data across wide-area links twice.

A non-coordinator-participant leader first acquires write locks. It then chooses a prepare timestamp that must be larger than any timestamps it has assigned to previous

transactions (to preserve monotonicity), and logs a prepare record through Paxos. Each participant then notifies the coordinator of its prepare timestamp.

The coordinator leader also first acquires write locks, but skips the prepare phase. It chooses a timestamp for the entire transaction after hearing from all other participant leaders. The commit timestamp s must be greater than or equal to all prepare timestamps (to satisfy the constraints discussed in Section 4.1.3), greater than $TT.now().latest$ at the time the coordinator received its commit message, and greater than any timestamps the leader has assigned to previous transactions (again, to preserve monotonicity). The coordinator leader then logs a commit record through Paxos (or an abort if it timed out while waiting for the other participants).

Only Paxos leaders acquire locks. Lock state is only logged during transaction prepare. If any locks have been lost before prepare (either due to deadlock avoidance, timeout, or a Paxos leader change), the participant aborts. Spanner ensures that a prepare or commit record is only logged while all locks are still held. In the event of a leader change, the new leader restores any lock state for prepared but uncommitted transactions before it accepts any new transactions.

Before allowing any coordinator replica to apply the commit record, the coordinator leader waits until $TT.after(s)$, so as to obey the commit-wait rule described in Section 4.1.2. Because the coordinator leader chose s based on $TT.now().latest$, and now waits until that timestamp is guaranteed to be in the past, the expected wait is at least $2 * \bar{\epsilon}$. This wait is typically overlapped with Paxos communication. After commit wait, the coordinator sends the commit timestamp to the client and all other participant leaders. Each participant leader logs the transaction's outcome through Paxos. All participants apply at the same timestamp and then release locks.

4.2.2. Snapshot Transactions. Assigning a timestamp requires a negotiation phase between all of the Paxos groups that are involved in the reads. As a result, Spanner requires a *scope* expression for every snapshot transaction, which is an expression that summarizes the keys that will be read by the entire transaction. Spanner automatically infers the scope for standalone queries.

If the scope's values are served by a single Paxos group, then the client issues the snapshot transaction to that group's leader. (The current Spanner implementation only chooses a timestamp for a snapshot transaction at a Paxos leader.) That leader assigns s_{read} and executes the read. For a single-site read, Spanner generally does better than $TT.now().latest$. Define $LastTS()$ to be the timestamp of the last committed write at a Paxos group. If there are no prepared transactions, the assignment $s_{read} = LastTS()$ trivially satisfies external consistency: the transaction will see the result of the last write, and therefore be ordered after it.

If the scope's values are served by multiple Paxos groups, there are several options. The most complicated option is to do a round of communication with all of the groups's leaders to negotiate s_{read} based on $LastTS()$. Spanner currently implements a simpler choice. The client avoids a negotiation round, and just executes its reads at $s_{read} = TT.now().latest$ (which may wait for safe time to advance). All reads in the transaction can be sent to replicas that are sufficiently up-to-date.

4.2.3. Schema-Change Transactions. TrueTime enables Spanner to support atomic schema changes. It would be infeasible to use a standard transaction, because the number of participants (the number of groups in a database) could be in the millions. Bigtable supports atomic schema changes in one datacenter, but its schema changes block all operations.

A Spanner schema-change transaction is a generally nonblocking variant of a standard transaction. First, it is explicitly assigned a timestamp in the future, which is registered in the prepare phase. As a result, schema changes across thousands

of servers can complete with minimal disruption to other concurrent activity. Second, reads and writes, which implicitly depend on the schema, synchronize with any registered schema-change timestamp at time t : they may proceed if their timestamps precede t , but they must block behind the schema-change transaction if their timestamps are after t . Without TrueTime, defining the schema change to happen at t would be meaningless.

4.2.4. Refinements. t_{safe}^{TM} as defined here has a weakness, in that a single prepared transaction prevents t_{safe} from advancing. As a result, no reads can occur at later timestamps, even if the reads do not conflict with the transaction. Such false conflicts can be removed by augmenting t_{safe}^{TM} with a fine-grained mapping from key ranges to prepared-transaction timestamps. This information can be stored in the lock table, which already maps key ranges to lock metadata. When a read arrives, it only needs to be checked against the fine-grained safe time for key ranges with which the read conflicts.

$LastTS()$ as defined here has a similar weakness: if a transaction has just committed, a nonconflicting snapshot transaction must still be assigned s_{read} so as to follow that transaction. As a result, the execution of the read could be delayed. This weakness can be remedied similarly by augmenting $LastTS()$ with a fine-grained mapping from key ranges to commit timestamps in the lock table. (We have not yet implemented this optimization.) When a snapshot transaction arrives, its timestamp can be assigned by taking the maximum value of $LastTS()$ for the key ranges with which the transaction conflicts, unless there is a conflicting prepared transaction (which can be determined from fine-grained safe time).

t_{safe}^{Paxos} as defined here has a weakness in that it cannot advance in the absence of Paxos writes. That is, a snapshot read at t cannot execute at Paxos groups whose last write happened before t . Spanner addresses this problem by taking advantage of the disjointness of leader-lease intervals. Each Paxos leader advances t_{safe}^{Paxos} by keeping a threshold above which future writes' timestamps will occur: it maintains a mapping $MinNextTS(n)$ from Paxos sequence number n to the minimum timestamp that may be assigned to Paxos sequence number $n + 1$. A replica can advance t_{safe}^{Paxos} to $MinNextTS(n) - 1$ when it has applied through n .

A single leader can enforce its $MinNextTS()$ promises easily. Because the timestamps promised by $MinNextTS()$ lie within a leader's lease, the disjointness invariant enforces $MinNextTS()$ promises across leaders. If a leader wishes to advance $MinNextTS()$ beyond the end of its leader lease, it must first extend its lease. Note that s_{max} is always advanced to the highest value in $MinNextTS()$ to preserve disjointness.

A leader by default advances $MinNextTS()$ values every 8 seconds. Thus, in the absence of prepared transactions, healthy slaves in an idle Paxos group can serve reads at timestamps greater than 8 seconds old in the worst case. A leader may also advance $MinNextTS()$ values on demand from slaves.

4.2.5. Paxos Leader-Lease Management. The simplest means to ensure the disjointness of Paxos-leader-lease intervals would be for a leader to issue a synchronous Paxos write of the lease interval, whenever it would be extended. A subsequent leader would read the interval and wait until that interval has passed.

TrueTime can be used to ensure disjointness without these extra log writes. The potential i th leader keeps a lower bound on the start of a lease vote from replica r as $v_{i,r}^{leader} = TT.now().earliest$, computed before $e_{i,r}^{send}$ (defined as when the lease

Table III. Operation Microbenchmarks. Mean and Standard Deviation over 10 Runs. 1D Means One Replica with Commit Wait Disabled

replicas	latency (ms)			throughput (Kops/sec)		
	write	snapshot transaction	snapshot read	write	snapshot transaction	snapshot read
1D	10.1 ± 0.3	—	—	4.2 ± 0.03	—	—
1	14.1 ± 1.0	1.3 ± 0.04	1.3 ± 0.02	4.2 ± 0.07	10.7 ± 0.6	11.4 ± 0.2
3	14.3 ± 0.5	1.4 ± 0.08	1.4 ± 0.08	1.8 ± 0.03	11.2 ± 0.4	32.0 ± 1.8
5	15.4 ± 2.3	1.4 ± 0.07	1.6 ± 0.04	1.2 ± 0.2	11.1 ± 0.5	46.8 ± 5.8

request is sent by the leader). Each replica r grants a lease at time $e_{i,r}^{grant}$, which happens after $e_{i,r}^{receive}$ (when the replica receives a lease request); the lease ends at $t_{i,r}^{end} = TT.now().latest + lease_length$, computed after $e_{i,r}^{receive}$. A replica r obeys the **single-vote** rule: it will not grant another lease vote until $TT.after(t_{i,r}^{end})$ is true. To enforce this rule across different incarnations of r , Spanner logs a lease vote at the granting replica before granting the lease; this log write can be piggybacked upon existing Paxos-protocol log writes.

When the i th leader receives a quorum of votes (event e_i^{quorum}), it computes its lease interval as $lease_i = [TT.now().latest, \min_r(v_{i,r}^{leader}) + lease_length]$. The lease is deemed to have expired at the leader when $TT.before(\min_r(v_{i,r}^{leader}) + lease_length)$ is false. To prove disjointness, we make use of the fact that the i th and $(i+1)$ th leaders must have one replica in common in their quorums. Call that replica r_0 . Proof:

$$\begin{aligned}
lease_i.end &= \min_r(v_{i,r}^{leader}) + lease_length && \text{(by definition)} \\
\min_r(v_{i,r}^{leader}) + lease_length &\leq v_{i,r_0}^{leader} + lease_length && \text{(min)} \\
v_{i,r_0}^{leader} + lease_length &\leq t_{abs}(e_{i,r_0}^{send}) + lease_length && \text{(by definition)} \\
t_{abs}(e_{i,r_0}^{send}) + lease_length &\leq t_{abs}(e_{i,r_0}^{receive}) + lease_length && \text{(causality)} \\
t_{abs}(e_{i,r_0}^{receive}) + lease_length &\leq t_{i,r_0}^{end} && \text{(by definition)} \\
t_{i,r_0}^{end} &< t_{abs}(e_{i+1,r_0}^{grant}) && \text{(single-vote)} \\
t_{abs}(e_{i+1,r_0}^{grant}) &\leq t_{abs}(e_{i+1}^{quorum}) && \text{(causality)} \\
t_{abs}(e_{i+1}^{quorum}) &\leq lease_{i+1}.start && \text{(by definition)}
\end{aligned}$$

5. EVALUATION

We first measure Spanner's performance with respect to replication, transactions, and availability. We then provide some data on TrueTime behavior, and a case study of our first client, F1.

5.1. Microbenchmarks

Table III presents some microbenchmarks for Spanner. These measurements were taken on timeshared machines: each spanserver ran on scheduling units of 4GB RAM and 4 cores (AMD Barcelona 2200MHz). Clients were run on separate machines. Each zone contained one spanserver. Clients and zones were placed in a set of datacenters with network distance of less than 1ms. (Such a layout should be commonplace: most

Table IV. Two-Phase Commit Scalability. Mean and Standard Deviations over 10 Runs

participants	latency (ms)	
	mean	99th percentile
1	14.6 ± 0.2	26.550 ± 6.2
2	20.7 ± 0.4	31.958 ± 4.1
5	23.9 ± 2.2	46.428 ± 8.0
10	22.8 ± 0.4	45.931 ± 4.2
25	26.0 ± 0.6	52.509 ± 4.3
50	33.8 ± 0.6	62.420 ± 7.1
100	55.9 ± 1.2	88.859 ± 5.9
200	122.5 ± 6.7	206.443 ± 15.8

applications do not need to distribute all of their data worldwide.) The test database was created with 50 Paxos groups with 2500 directories. Operations were standalone reads and writes of 4KB. All reads were served out of memory after a compaction, so that we are only measuring the overhead of Spanner's call stack. In addition, one unmeasured round of reads was done first to warm any location caches.

For the latency experiments, clients issued sufficiently few operations so as to avoid queuing at the servers. From the 1-replica experiments, commit wait is about 4ms, and Paxos latency is about 10ms. As the number of replicas increases, the latency increases slightly because Paxos must commit at more replicas. Because Paxos executes in parallel at a group's replicas, the increase is sublinear: the latency depends on the slowest member of the quorum. We can see the most measurement noise in write latency.

For the throughput experiments, clients issued sufficiently many operations so as to saturate the servers' CPUs. In addition, Paxos leaders were pinned to one zone, so as to keep constant across experiments, the number of machines on which leaders run. Snapshot reads can execute at any up-to-date replicas, so their throughput increases with the number of replicas. Single-read snapshot transactions only execute at leaders because timestamp assignment must happen at leaders. As a result, snapshot-transaction throughput stays roughly constant with the number of replicas. Finally, write throughput decreases with the number of replicas, since the amount of Paxos work increases linearly with the number of replicas. Again, there is a fair amount of noise in our measurements, since we ran in production datacenters: we had to discard some data that could only be explained by interference from other jobs.

Table IV demonstrates that two-phase commit can scale to a reasonable number of participants: it summarizes a set of experiments run across 3 zones, each with 25 spanservers. Scaling up to 50 participants is reasonable in both mean and 99th-percentile; latencies start to rise noticeably at 100 participants.

5.2. Availability

Figure 5 illustrates the availability benefits of running Spanner in multiple datacenters. It shows the results of three experiments on throughput in the presence of datacenter failure, all of which are overlaid onto the same time scale. The test universe consisted of 5 zones Z_i , each of which had 25 spanservers. The test database was sharded into 1250 Paxos groups, and 100 test clients constantly issued nonsnapshot reads at an aggregate rate of 50K reads/second. All of the leaders were explicitly placed in Z_1 . Five seconds into each test, all of the servers in one zone were killed: *nonleader* kills Z_2 ; *leader-hard* kills Z_1 ; *leader-soft* kills Z_1 , but it gives notifications to all of the servers that they should handoff leadership first.

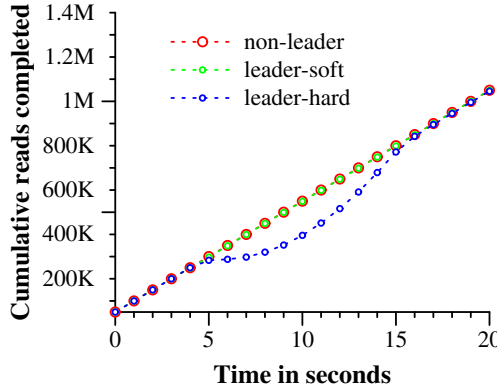


Fig. 5. Effect of killing servers on throughput.

Killing Z_2 does not affect read throughput. Killing Z_1 while giving the leaders time to handoff leadership to a different zone has a minor effect: the throughput drop is not visible in the graph, but is around 3–4%. On the other hand, killing Z_1 with no warning has a severe effect: the rate of completion drops almost to 0. As leaders get reelected, though, the throughput of the system rises to approximately 100K reads/second because of two artifacts of our experiment: there is extra capacity in the system, and operations are queued while the leader is unavailable. As a result, the throughput of the system rises before leveling off again at its steady-state rate.

We can also see the effect of the fact that Paxos leader leases are set to 10 seconds. When we kill the zone, the leader-lease expiration times for the groups should be evenly distributed over the next 10 seconds. Soon after each lease from a dead leader expires, a new leader is elected. Approximately 10 seconds after the kill time, all of the groups have leaders and throughput has recovered. Shorter lease times would reduce the effect of server deaths on availability, but would require greater amounts of lease-renewal network traffic. We are in the process of designing and implementing a mechanism that will cause slaves to release Paxos leader leases upon leader failure.

5.3. TrueTime

Two questions must be answered with respect to TrueTime: is ϵ truly a bound on clock uncertainty, and how bad does ϵ get? For the former, the most serious problem would be if a local clock's drift were greater than 200us/sec: that would break assumptions made by TrueTime. Our machine statistics show that bad CPUs are 6 times more likely than bad clocks. That is, clock issues are extremely infrequent, relative to much more serious hardware problems. As a result, we believe that TrueTime's implementation is as trustworthy as any other piece of software upon which Spanner depends.

Figure 6 presents TrueTime data taken at several thousand spanserver machines across datacenters up to 2200 km apart. It plots the 90th, 99th, and 99.9th percentiles of ϵ , sampled at timeslave daemons immediately after polling the time masters. This sampling elides the sawtooth in ϵ due to local-clock uncertainty, and therefore measures time-master uncertainty (which is generally 0) plus communication delay to the time masters.

The data shows that these two factors in determining the base value of ϵ are generally not a problem. However, there can be significant tail-latency issues that cause higher values of ϵ . The reduction in tail latencies beginning on March 30 were due to networking improvements that reduced transient network-link congestion. The increase in ϵ on April 13, approximately one hour in duration, resulted from the

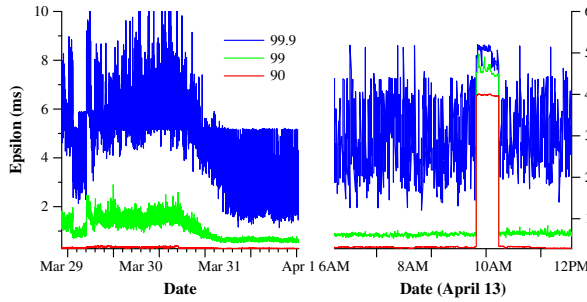


Fig. 6. Distribution of TrueTime ϵ values, sampled right after timeslave daemon polled the time masters. 90th, 99th, and 99.9th percentiles are graphed.

shutdown of 2 time masters at a datacenter for routine maintenance. We continue to investigate and remove causes of TrueTime spikes.

5.4. F1

Spanner started being experimentally evaluated under production workloads in early 2011, as part of a rewrite of Google's advertising backend, called F1 [Shute et al. 2012]. This backend was originally based on a MySQL database that was manually sharded many ways. The uncompressed dataset is tens of terabytes, which is small compared to many NoSQL instances, but was large enough to cause difficulties with sharded MySQL. The MySQL sharding scheme assigned each customer and all related data to a fixed shard. This layout enabled the use of indexes and complex query processing on a per-customer basis, but required some knowledge of the sharding in application business logic. Resharding this revenue-critical database as it grew in the number of customers and the size of their data was extremely costly. The last resharding took over two years of intense effort, and involved coordination and testing across dozens of teams to minimize risk. This operation was too complex to do regularly: as a result, the team had to limit growth on the MySQL database by storing some data in external Bigtables, which compromised transactional behavior and the ability to query across all data.

The F1 team chose to use Spanner for several reasons. First, Spanner removes the need to manually reshard. Second, Spanner provides synchronous replication and automatic failover. With MySQL master-slave replication, failover was difficult, and risked data loss and downtime. Third, F1 requires strong transactional semantics, which made using other NoSQL systems impractical. Application semantics requires transactions across arbitrary data, and consistent reads. The F1 team also needed secondary indexes on their data (since Spanner does not yet provide automatic support for secondary indexes), and was able to implement their own consistent global indexes using Spanner transactions.

All application writes are now by default sent through F1 to Spanner, instead of the MySQL-based application stack. F1 has 2 replicas on the west coast of the US, and 3 on the east coast. This choice of replica sites was made to cope with outages due to potential major natural disasters, and also the choice of their frontend sites. Anecdotally, Spanner's automatic failover has been nearly invisible to them. Although there have been unplanned cluster failures in the last few months, the most that the F1 team has had to do is update their database's schema to tell Spanner where to preferentially place Paxos leaders, so as to keep them close to where their frontends moved.

Table V. Distribution of Directory-Fragment Counts in F1

# fragments	# directories
1	>100M
2–4	341
5–9	5336
10–14	232
15–99	34
100–500	7

Table VI. F1-Perceived Operation Latencies Measured over the Course of 24 Hours

operation	latency (ms)		count
	mean	std dev	
all reads	8.7	376.4	21.5B
single-site commit	72.3	112.8	31.2M
multi-site commit	103.0	52.2	32.1M

Spanner's timestamp semantics made it efficient for F1 to maintain in-memory data structures computed from the database state. F1 maintains a logical history log of all changes, which is written into Spanner itself as part of every transaction. F1 takes full snapshots of data at a timestamp to initialize its data structures, and then reads incremental changes to update them.

Table V illustrates the distribution of the number of fragments per directory in F1. Each directory typically corresponds to a customer in the application stack above F1. The vast majority of directories (and therefore customers) consist of only 1 fragment, which means that reads and writes to those customers' data are guaranteed to occur on only a single server. The directories with more than 100 fragments are all tables that contain F1 secondary indexes: writes to more than a few fragments of such tables are extremely uncommon. The F1 team has only seen such behavior when they do untuned bulk data loads as transactions.

Table VI presents Spanner operation latencies as measured from F1 servers. Replicas in the east coast data centers are given higher priority in choosing Paxos leaders. The data in the table is measured from F1 servers in those data centers. The large standard deviation in write latencies is caused by a pretty fat tail due to lock conflicts. The even larger standard deviation in read latencies is partially due to the fact that Paxos leaders are spread across two data centers, only one of which has machines with SSDs. In addition, the measurement includes every read in the system from two datacenters: the mean and standard deviation of the bytes read were roughly 1.6KB and 119KB, respectively.

6. RELATED WORK

Consistent replication across datacenters as a storage service has been provided by Megastore [Baker et al. 2011] and DynamoDB [Amazon 2012]. DynamoDB presents a key-value interface, and only replicates within a region. Spanner follows Megastore in providing a semirelational data model, and even a similar schema language. Megastore does not achieve high performance. It is layered on top of Bigtable, which imposes high communication costs. It also does not support long-lived leaders: multiple replicas may initiate writes. All writes from different replicas necessarily conflict in the Paxos protocol, even if they do not logically conflict: throughput collapses on a Paxos group

at several writes per second. Spanner provides higher performance, general-purpose transactions, and external consistency.

Pavlo et al. [2009] have compared the performance of databases and MapReduce [Dean and Ghemawat 2010]. They point to several other efforts that have been made to explore database functionality layered on distributed key-value stores [Abouzeid et al. 2009; Armbrust et al. 2011; Brantner et al. 2008; Thusoo et al. 2010] as evidence that the two worlds are converging. We agree with the conclusion, but demonstrate that integrating multiple layers has its advantages: integrating concurrency control with replication reduces the cost of commit wait in Spanner, for example.

The notion of layering transactions on top of a replicated store dates at least as far back as Gifford's dissertation [Gifford 1982]. Scatter [Glendenning et al. 2011] is a recent DHT-based key-value store that layers transactions on top of consistent replication. Spanner focuses on providing a higher-level interface than Scatter does. Gray and Lamport [2006] describe a nonblocking commit protocol based on Paxos. Their protocol incurs more messaging costs than two-phase commit, which would aggravate the cost of commit over widely distributed groups. Walter [Sovran et al. 2011] provides a variant of snapshot isolation that works within, but not across, datacenters. In contrast, our snapshot transactions provide a more natural semantics, because we support external consistency over all operations.

There has been a spate of recent work on reducing or eliminating locking overheads. Calvin [Thomson et al. 2012] eliminates concurrency control: it preassigns timestamps and then executes the transactions in timestamp order. H-Store [Stonebraker et al. 2007] and Granola [Cowling and Liskov 2012] each supported their own classification of transaction types, some of which could avoid locking. None of these systems provides external consistency. Spanner addresses the contention issue by providing support for snapshot isolation.

VoltDB [2012] is a sharded in-memory database that supports master-slave replication over the wide area for disaster recovery, but not more general replication configurations. It is an example of what has been called NewSQL, which is a marketplace push to support scalable SQL [Stonebraker 2010a]. A number of commercial databases implement reads in the past, such as MarkLogic [2012] and Oracle's Total Recall [Oracle 2012]. Lomet and Li [2009] describe an implementation strategy for such a temporal database.

Farsite derived bounds on clock uncertainty (much looser than TrueTime's) relative to a trusted clock reference [Douceur and Howell 2003]: server leases in Farsite were maintained in the same way that Spanner maintains Paxos leases. Loosely synchronized clocks have been used for concurrency-control purposes in prior work [Adya et al. 1995; Liskov 1993]. We have shown that TrueTime lets one reason about global time across sets of Paxos state machines.

7. FUTURE WORK

We have spent most of the last year working with the F1 team to transition Google's advertising backend from MySQL to Spanner. We are actively improving its monitoring and support tools, as well as tuning its performance. In addition, we have been working on improving the functionality and performance of our backup/restore system. We are currently implementing the Spanner schema language, automatic maintenance of secondary indices, and automatic load-based resharding. Longer term, there are a couple of features that we plan to investigate. Optimistically doing reads in parallel may be a valuable strategy to pursue, but initial experiments have indicated that the right implementation is nontrivial. In addition, we plan to eventually support direct changes of Paxos configurations [Lamport et al. 2010; Shraer et al. 2012].

Given that we expect many applications to replicate their data across datacenters that are relatively close to each other, TrueTime ϵ may noticeably affect performance. We see no insurmountable obstacle to reducing ϵ below 1ms. Time-master-query intervals can be reduced, and better clock crystals are relatively cheap. Time-master query latency could be reduced with improved networking technology, or possibly even avoided through alternate time-distribution technology.

Finally, there are obvious areas for improvement. Although Spanner is scalable in the number of nodes, the node-local data structures have relatively poor performance on complex SQL queries, because they were designed for simple key-value accesses. Algorithms and data structures from DB literature could improve single-node performance a great deal. Second, moving data automatically between datacenters in response to changes in client load has long been a goal of ours, but to make that goal effective, we would also need the ability to move client-application processes between datacenters in an automated, coordinated fashion. Moving processes raises the even more difficult problem of managing resource acquisition and allocation between datacenters.

8. CONCLUSIONS

To summarize, Spanner combines and extends ideas from two research communities: from the database community, a familiar, easy-to-use, semirelational interface, transactions, and an SQL-based query language; from the systems community, scalability, automatic sharding, fault tolerance, consistent replication, external consistency, and wide-area distribution. Since Spanner's inception, we have taken more than 5 years to iterate to the current design and implementation. Part of this long iteration phase was due to a slow realization that Spanner should not only tackle the problem of a globally replicated namespace, it should also focus on database features that Bigtable was missing.

One aspect of our design stands out: the linchpin of Spanner's feature set is TrueTime. We have shown that reifying clock uncertainty in the time API makes it possible to build distributed systems with much stronger time semantics. In addition, as the underlying system enforces tighter bounds on clock uncertainty, the overhead of the stronger semantics decreases. As a community, we should no longer depend on loosely synchronized clocks and weak time APIs in designing distributed algorithms.

ACKNOWLEDGMENTS

Many people have helped to improve this article: our shepherd Jon Howell, who went above and beyond his responsibilities; the anonymous referees; and many Googlers: Atul Adya, Fay Chang, Frank Dabek, Sean Dorward, Bob Gruber, David Held, Nick Kline, Alex Thomson, and Joel Wein. Our management has been very supportive of both our work and of publishing this article: Aristotle Balogh, Bill Coughran, Urs Hölzle, Doron Meyer, Cos Nicolaou, Kathy Polizzi, Sridhar Ramaswamy, and Shivakumar Venkataraman. We have built upon the work of the Bigtable and Megastore teams. The F1 team, and Jeff Shute in particular, worked closely with us in developing our data model and helped immensely in tracking down performance and correctness bugs. The Platforms team, and Luiz Barroso and Bob Felderman in particular, helped to make TrueTime happen. Finally, a lot of Googlers used to be on our team: Ken Ashcraft, Paul Cychosz, Krzysztof Ostrowski, Amir Voskoboinik, Matthew Weaver, Theo Vassilakis, and Eric Veach; or have joined our team recently: Nathan Bales, Adam Beberg, Vadim Borisov, Ken Chen, Brian Cooper, Cian Cullinan, Robert-Jan Huijsman, Milind Joshi, Andrey Khorlin, Dawid Kuroczko, Laramie Leavitt, Eric Li, Mike Mammarella, Sunil Mushran, Simon Nielsen, Ovidiu Platon, Ananth Shrinivas, Vadim Suvorov, and Marcel van der Holst.

REFERENCES

- Abouzeid, A., Bajda-Pawlikowski, K., Abadi, D., Silberschatz, A., and Rasin, A. 2009. Hadoopdb: An architectural hybrid of mapreduce and DBMS technologies for analytical workloads. In *Proceedings of the International Conference on Very Large Data Bases*. 922–933.

- Adya, A., Gruber, R., Liskov, B., and Maheshwari, U. 1995. Efficient optimistic concurrency control using loosely synchronized clocks. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 23–34.
- Amazon. 2012. Amazon dynamodb. <http://aws.amazon.com/dynamodb>.
- Armbrust, M., Curtis, K., Kraska, T., Fox, A., Franklin, M., and Patterson, D. 2011. PIQL: Success-tolerant query processing in the cloud. In *Proceedings of the International Conference on Very Large Data Bases*. 181–192.
- Baker, J., Bond, C., Corbett, J. C., Furman, J., Khorlin, A., Larson, J., Léon, J.-M., Li, Y., Lloyd, A., and Yushprakh, V. 2011. Megastore: Providing scalable, highly available storage for interactive services. In *Proceedings of CIDR*. 223–234.
- Berenson, H., Bernstein, P., Gray, J., Melton, J., O’Neil, E., and O’Neil, P. 1995. A critique of ANSI SQL isolation levels. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 1–10.
- Brantner, M., Florescu, D., Graf, D., Kossman, D., and Kraska, T. 2008. Building a database on S3. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 251–264.
- Chan, A. and Gray, R. 1985. Implementing distributed read-only transactions. *IEEE Trans. Softw. Eng. SE-11*, 2, 205–212.
- Chang, F., Dean, J., Ghemawat, S., Hsieh, W. C., Wallach, D. A., Burrows, M., Chandra, T., Fikes, A., and Gruber, R. E. 2008. Bigtable: A distributed storage system for structured data. *ACM Trans. Comput. Syst.* 26, 2, 4:1–4:26.
- Cooper, B. F., Ramakrishnan, R., Srivastava, U., Silberstein, A., Bohannon, P., Jacobsen, H.-A., Puz, N., Weaver, D., and Yerneni, R. 2008. PNUTS: Yahoo!’s hosted data serving platform. In *Proceedings of the International Conference on Very Large Data Bases*. 1277–1288.
- Cowling, J. and Liskov, B. 2012. Granola: Low-overhead distributed transaction coordination. In *Proceedings of USENIX ATC*. 223–236.
- Dean, J. and Ghemawat, S. 2010. MapReduce: A flexible data processing tool. *Comm. ACM* 53, 1, 72–77.
- Douceur, J. and Howell, J. 2003. Scalable Byzantine-fault-quantifying clock synchronization. Tech. rep. MSR-TR-2003-67, MS Research.
- Douceur, J. R. and Howell, J. 2006. Distributed directory service in the Farsite file system. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation*. 321–334.
- Ghemawat, S., Gobiolf, H., and Leung, S.-T. 2003. The Google file system. In *Proceedings of the Symposium on Operating Systems Principles*. 29–43.
- Gifford, D. K. 1982. Information storage in a decentralized computer system. Tech. rep. CSL-81-8, Xerox PARC.
- Glendenning, L., Beschastnikh, I., Krishnamurthy, A., and Anderson, T. 2011. Scalable consistency in scatter. In *Proceedings of the Symposium on Operating Systems Principles*.
- Google. 2008. Protocol buffers — Google’s data interchange format. <https://code.google.com/p/protobuf/>.
- Gray, J. and Lampert, L. 2006. Consensus on transaction commit. *ACM Trans. Datab. Syst.* 31, 1, 133–160.
- Helland, P. 2007. Life beyond distributed transactions: An apostate’s opinion. In *Proceedings of the Biennial Conference on Innovative Data Systems Research*. 132–141.
- Herlihy, M. P. and Wing, J. M. 1990. Linearizability: A correctness condition for concurrent objects. *ACM Trans. Program. Lang. Syst.* 12, 3, 463–492.
- Lamport, L. 1998. The part-time parliament. *ACM Trans. Comput. Syst.* 16, 2, 133–169.
- Lamport, L., Malkhi, D., and Zhou, L. 2010. Reconfiguring a state machine. *SIGACT News* 41, 1, 63–73.
- Liskov, B. 1993. Practical uses of synchronized clocks in distributed systems. *Distrib. Comput.* 6, 4, 211–219.
- Lomet, D. B. and Li, F. 2009. Improving transaction-time DBMS performance and functionality. In *Proceedings of the International Conference on Data Engineering*. 581–591.
- Lorch, J. R., Adya, A., Bolosky, W. J., Chaiken, R., Douceur, J. R., and Howell, J. 2006. The SMART way to migrate replicated stateful services. In *Proceedings of EuroSys*. 103–115.
- MarkLogic. 2012. Marklogic 5 product documentation. <http://community.marklogic.com/docs>.
- Marzullo, K. and Owicki, S. 1983. Maintaining the time in a distributed system. In *Proceedings of the Symposium on Principles of Distributed Computing*. 295–305.
- Melnik, S., Gubarev, A., Long, J. J., Romer, G., Shivakumar, S., Tolton, M., and Vassilakis, T. 2010. Dremel: Interactive analysis of Web-scale datasets. In *Proceedings of the International Conference on Very Large Data Bases*. 330–339.
- Mills, D. 1981. Time synchronization in DCNET hosts. Internet project rep. IEN-173, COMSAT Laboratories.

- Oracle. 2012. Oracle total recall. <http://www.oracle.com/technetwork/database/focus-areas/storage/total-recall-whitepaper-171749.pdf>.
- Pavlo, A., Paulson, E., Rasin, A., Abadi, D. J., DeWitt, D. J., Madden, S., and Stonebraker, M. 2009. A comparison of approaches to large-scale data analysis. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 165–178.
- Peng, D. and Dabek, F. 2010. Large-scale incremental processing using distributed transactions and notifications. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation*. 1–15.
- Rosenkrantz, D. J., Stearns, R. E., and Lewis II, P. M. 1978. System level concurrency control for distributed database systems. *ACM Trans. Datab. Syst.* 3, 2, 178–198.
- Shraer, A., Reed, B., Malkhi, D., and Junqueira, F. 2012. Dynamic reconfiguration of primary/backup clusters. In *Proceedings of USENIX ATC*. 425–438.
- Shute, J., Oancea, M., Ellner, S., Handy, B., Rollins, E., Samwel, B., Vingralek, R., Whipkey, C., Chen, X., Jegerlehner, B., Littlefield, K., and Tong, P. 2012. F1 — The fault-tolerant distributed RDBMS supporting Google’s ad business. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 777–778.
- Sovran, Y., Power, R., Aguilera, M. K., and Li, J. 2011. Transactional storage for geo-replicated systems. In *Proceedings of the Symposium on Operating Systems Principles*. 385–400.
- Stonebraker, M. 2010a. Six SQL urban myths. <http://highscalability.com/blog/2010/6/28/voltdb-decapitates-six-sql-urban-myths-and-delivers-internet.html>.
- Stonebraker, M. 2010b. Why enterprises are uninterested in NoSQL. <http://cacm.acm.org/blogs/blog-cacm/99512-why-enterprises-are-uninterested-in-nosql/fulltext>.
- Stonebraker, M., Madden, S., Abadi, D. J., Harizopoulos, S., Hachem, N., and Helland, P. 2007. The end of an architectural era: (It’s time for a complete rewrite). In *Proceedings of the International Conference on Very Large Data Bases*. 1150–1160.
- Thomson, A., Diamond, T., Shu-Chun Weng, T. D., Ren, K., Shao, P., and Abadi, D. J. 2012. Calvin: Fast distributed transactions for partitioned database systems. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 1–12.
- Thusoo, A., Sarma, J. S., Jain, N., Shao, Z., Chakka, P., Zhang, N., Antony, S., Liu, H., and Murthy, R. 2010. Hive — A petabyte scale data warehouse using Hadoop. In *Proceedings of the International Conference on Data Engineering*. 996–1005.
- VoltDB. 2012. VoltDB resources. <http://voltdb.com/resources/whitepapers>.

Received January 2013; accepted May 2013